

Security Threats in AI-Native Operating Systems: ~~An Empirical Study~~ Using Privilege-Escalated LLM Agents

A Platform and Threat Framework for

Yakup Atahanov, Toufic Majdalani, **Jeremy Thompson**
Washington State University Everett

Abstract

As Large Language Models (LLMs) transition from conversational tools to autonomous agents with system-level control, critical security questions remain unanswered: What threats emerge when AI operates with elevated privileges, and how can they be mitigated? Despite growing AI deployment in critical systems, empirical research on LLM behavior with unrestricted OS access—and frameworks to study it—remains limited.

This paper presents JarvisOS, an Arch Linux-based AI-native operating system and open-source security testbed featuring a dynamic Model Context Protocol (MCP) orchestration layer. Notably, **JarvisOS implemented a structured MCP tool-description architecture in October–November 2025, a design pattern later adopted independently by major AI platforms in early 2026.** Through developing and operating this system, we empirically identified six security threats that emerge when LLMs are granted elevated privileges: Malicious MCP Servers, Prompt Injection, Misleading MCP Server Usage, Unauthorized Sudo Requests via MCP, Sudo Capability Exploitation, and Bloated Context—a phenomenon where context window saturation causes LLMs to silently discard security constraints and exhibit unpredictable behavior. We evaluate these threats across three escalation stages: (1) user-level privileges, (2) sudo access, and (3) web-enabled operations, and present architectural mitigations for each threat class.

JarvisOS is released as an open-source platform at <https://github.com/YakupAtahanov/Project-JARVIS>.

Index Terms—AI Security, LLM Agents, Privilege Escalation, Model Context Protocol, Operating Systems Security, Prompt Injection

1. INTRODUCTION

The rapid evolution of Large Language Models (LLMs) has fundamentally expanded their role in computing. Once confined to text generation and question answering, modern LLMs now serve as autonomous agents capable of planning multi-step tasks, invoking external tools, and executing actions with real-world consequences. As this capability expands, so does the attack surface they introduce—particularly when these agents are granted elevated system privileges.

Operating systems represent one of the highest-stakes environments in which AI agents can operate. An LLM with unrestricted access to a system—able to read and write files, execute processes, and invoke privileged commands—poses qualitatively different security risks than a sandboxed chatbot. Yet despite this, empirical research on LLM behavior at the OS level remains scarce. Most existing security work on LLMs focuses on model-level attacks such as jailbreaking or adversarial inputs, rather than on the systemic threats that emerge

when an LLM agent is placed in control of a real computing environment.

This paper addresses that gap. We present JarvisOS—an Arch Linux-based, AI-native operating system built as both a usable platform and an empirical research testbed for studying the security behavior of privilege-escalated LLM agents. Through designing, building, and operating JarvisOS, we identified, reproduced, and mitigated six distinct threat classes that arise specifically at the intersection of LLM agency and OS-level privilege. For each threat, we present not only the attack vector, but a concrete architectural mitigation.

Beyond its research contributions, JarvisOS is released as open-source software under a dual-license model, inviting cybersecurity researchers, developers, and the broader community to build on, stress-test, and extend this platform.

1.1 How MCP Servers Work

The Model Context Protocol (MCP) is an open standard that defines how LLMs can invoke external tools and services in a structured, composable way. Rather than relying on the LLM’s internal knowledge alone, MCP enables it to take actions in the real world—reading files, querying databases, calling APIs, or executing system commands—by delegating those operations to purpose-built MCP servers.

An MCP server is a self-contained process that exposes a defined set of tools. Each tool has a name, a description, and an input schema. When an LLM receives a user request, it determines whether any available tool is relevant, constructs a structured call to that tool, receives the result, and incorporates it into its response. Critically, the tools a server exposes must be predefined—the LLM cannot invoke capabilities that have not been explicitly implemented and declared.

A simple analogy: without MCP, an LLM is like an assistant who can only give advice. With MCP, that same assistant can also send emails, open files, and run programs on your behalf—but only the specific programs and actions their toolset was built to support. The boundary of what they can do is defined entirely by the servers they have access to.

In JarvisOS, MCP servers form the core action layer. The system dynamically discovers and loads relevant servers using a live vector and keyword search engine within dmcp, ensuring the LLM only loads tools it actually needs—rather than bloating the context with every available capability at once. This architecture was first implemented in JarvisOS in October–November

2025, predating its independent adoption by major AI platforms in early 2026.

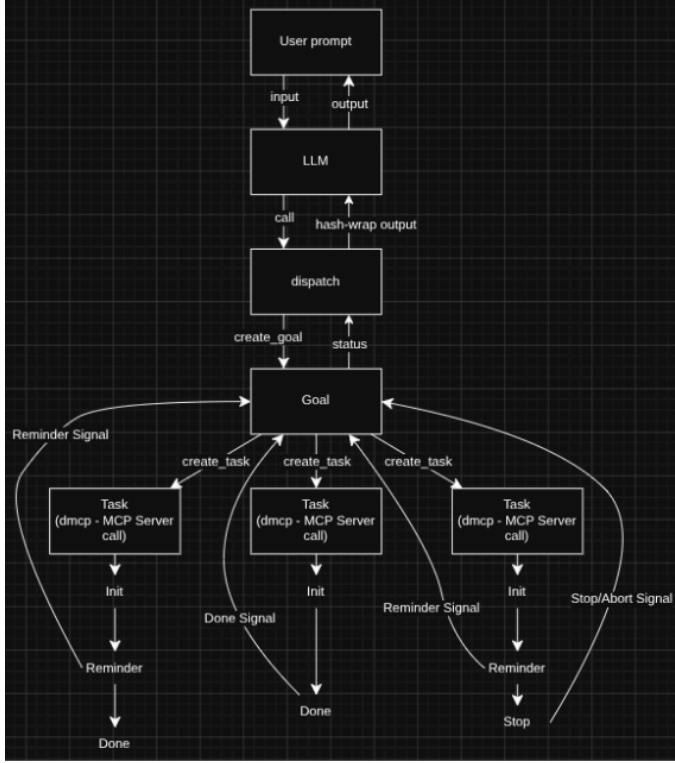


Figure 1: MCP request flow in JarvisOS. User input enters at the ROOT level and is routed to the DISPATCH sub-chain, which performs semantic tool discovery via the dmcp vector index. The dispatch Rust orchestrator spawns tasks concurrently, tags each result with a provenance nonce, and wakes the LLM only on signal events. Successful MCP output is stored out-of-band and does not appear in the signal window unless explicitly retrieved.

1.2 Motivation for an AI-Native Operating System

The central question motivating this research is not specific to operating systems: it is what security threats emerge when an LLM is integrated into any real system with actionable capabilities. As LLMs are increasingly embedded into applications, developer tools, automation pipelines, and infrastructure—not merely as advisors but as agents that can take actions—understanding their threat profile becomes critical.

We selected an operating system as our research environment deliberately, not incidentally. An OS represents the broadest possible integration surface: it encompasses file and directory management, process execution, network operations, package installation, privilege escalation, and system configuration. By studying LLM agency at the OS level, we maximize the diversity of threat scenarios we can observe. Threats that might remain latent in a narrower integration—say, an LLM embedded only in a text editor or a web browser—surface naturally when the agent has access to the full capability stack of an operating system.

JarvisOS is therefore best understood as a worst-case test environment by design. If we can characterize and mitigate the threats that emerge here, the findings generalize downward to less privileged integration contexts. The OS is the upper bound; the lessons it yields apply broadly.

1.3 Research Questions and Objectives

This paper is organized around two central research questions:

- **RQ1:** What distinct security threats emerge when an LLM agent is granted elevated operating system privileges, including sudo access and web-enabled operations?
- **RQ2:** How can each identified threat class be mitigated through architectural design decisions in an AI-native OS?

To address these questions, we designed and built JarvisOS as a controlled empirical testbed, operated it under three escalating privilege conditions—user-level, sudo-enabled, and web-enabled—and systematically documented the security threats that emerged at each stage. For each threat, we developed and validated an architectural mitigation that is now implemented in the platform.

The concrete contributions of this paper are:

- A taxonomy of six empirically-identified security threats specific to privilege-escalated LLM agents.
- Architectural mitigations for each threat class, implemented and validated in JarvisOS.
- JarvisOS itself—a fully functional, open-source AI-native OS released as a research and development platform for the community.
- A documented MCP tool-description architecture, independently developed in October–November 2025, predating its adoption by major AI platforms.

2. RELATED WORK

The security landscape surrounding privilege-escalated LLM agents sits at the intersection of several active research areas: prompt injection and adversarial attacks on LLMs, agentic AI security, retrieval and tool-selection mechanisms, and the emerging threat surface introduced by the Model Context Protocol. This section surveys each area and positions JarvisOS’s contributions within them.

2.1 Prompt Injection and Adversarial Attacks

Prompt injection has emerged as one of the most well-documented attack vectors against LLM-based systems. A 2025 systematic review synthesizing 128 peer-reviewed studies from 2022 to 2025 found a rapid progression from simple direct injections to sophisticated multimodal attacks, with success rates exceeding 90% against unprotected systems [1]. Attacks fall broadly into two categories: direct injection, where malicious instructions are embedded in user-facing prompts, and indirect

injection, where harmful content is introduced through external data sources the model retrieves or processes. Indirect prompt injection is of particular concern in LLM-integrated applications, where an attacker can embed instructions in external content—such as websites, documents, or API responses—causing the model to behave as the attacker intends rather than the user [2]. Defense research has explored input preprocessing, output validation, and model-level self-protection [3]. However, a single system prompt warning against injection is widely regarded as insufficient against determined adversaries—a finding that directly motivates JarvisOS’s architectural enforcement approach rather than prompt-level mitigations alone.

2.2 Agentic AI and Privilege Escalation

The extension of LLMs into agentic settings—where models plan, execute multi-step tasks, and invoke external tools—introduces a qualitatively different threat surface. A 2025 systematization maps the trust boundaries and security risks of agentic LLM systems, developing a comprehensive taxonomy spanning prompt-level injections, knowledge-base poisoning, tool and plugin exploits, and multiagent emergent threats [4]. The non-deterministic nature of LLM agents—where behavior is determined at runtime by natural language prompts rather than static code—enables a new class of privilege escalation attacks that existing system security models were not designed to address [5]. The transition from isolated LLM agents to autonomous systems introduces novel exploitation vectors including impersonation, task tampering, and unauthorized privilege escalation that go beyond traditional software security assumptions [7]. Research evaluating privilege usage of agents on real-world tools finds that LLMs often lack sufficient security awareness regarding privilege boundaries, and when facing malicious attacks, may abuse granted privileges in ways that lead to sensitive information leakage or destruction of critical infrastructure [6]. These findings corroborate and extend the threat classes identified empirically through JarvisOS.

2.3 Retrieval-Augmented Generation, Embeddings, and Tool Discovery

A central technical component of JarvisOS is its dynamic MCP tool discovery engine, which uses a hybrid of vector and keyword search within dmcp to match user intent against available tools at runtime. RAG was introduced to address the knowledge limitations of static LLMs, augmenting model responses with relevant content retrieved from external knowledge bases at inference time. The standard RAG pipeline encodes documents as dense vector embeddings, converts queries into the same embedding space, and retrieves the top-K most semantically similar results. Vector search transforms both documents and queries into high-dimensional numerical representations, then retrieves stored vectors most similar to the query using approximate nearest neighbor algorithms—enabling fast, semantically meaningful retrieval across large corpora. In JarvisOS, this mechanism is applied to MCP tool descriptions, allowing the

system to semantically match user requests to relevant tools without loading all available tools into context simultaneously.

Hybrid retrieval pipelines combining dense vector search with classical keyword search (BM25) have been shown to outperform either method alone—keyword search excels at exact term matching, while vector search captures semantic relationships and handles paraphrase and ambiguity. Recent work has validated this architecture specifically in the MCP context: indexing tools using dense embeddings and performing similarity search at query time achieves a 99.6% reduction in tool-related token consumption, a hit rate of 97.1% at K=3, and sub-100ms retrieval latency compared to approaches that inject the entire tool catalog into the context [10]. JarvisOS implemented this architecture independently in October–November 2025, predating this published result.

RAG systems are themselves a security surface. Knowledge corruption attacks against RAG systems demonstrate that injecting crafted malicious texts into the knowledge base can cause the system to generate attacker-specified responses for targeted queries [8]. Detection-based defenses such as chunk-wise perplexity filtering and text similarity filtering have shown effectiveness against poisoning attacks, though advanced adaptive attacks can still evade standard defenses [9]. These findings motivated JarvisOS’s use of a community-vetted, proofread MCP registry as a supply-chain control upstream of the runtime retrieval layer.

2.4 MCP Security

The Model Context Protocol, released by Anthropic in late 2024, has rapidly become the dominant standard for connecting LLM agents to external tools and services—and an increasingly studied attack surface. MCP introduces three classes of adversaries: content-injection attackers who embed malicious instructions in legitimate data, supply-chain attackers who distribute compromised servers, and agents that become unintentional adversaries by overstepping their intended role [11]. Supply-chain risks are particularly acute. The first malicious MCP package appeared in September 2025, operating undetected for two weeks while exfiltrating email data [15]. A critical remote code execution vulnerability (CVE-2025-49596, CVSS 9.4) was subsequently disclosed in Anthropic’s official MCP Inspector tool [13]. Analysis of 67,057 servers across six public MCP registries found that hosts perform no verification of model outputs, and a substantial number of servers were found susceptible to hijacking due to the lack of a vetted submission process [12]. Research on MCP marketplace security found that 9 out of 11 tested registries could be successfully poisoned with a malicious trial package [14]. These findings independently validate JarvisOS’s proofread registry model and provenance nonce protocol as necessary architectural responses.

2.5 Context Window Saturation and Bloated Context

The threat JarvisOS terms **Bloated Context**—where a saturated context window causes the LLM to silently discard earlier content, including security constraints—is supported by a growing body of research on context window behavior, though its specific security implications in agentic systems remain understudied. A model’s context window operates as a first-in, first-out buffer: when input and output tokens exceed window capacity, earlier information is silently displaced [16]. Research across 18 frontier models confirmed that effective context often falls far below advertised limits due to attention degradation and KV cache constraints [17]. Performance drops as context grows, particularly when relevant information is dispersed—a phenomenon referred to as being “lost in the middle,” where content placed centrally in a long context receives less attention weight than content near the edges [18].

The security-specific implication—that security constraints placed in a system prompt may be effectively ignored as the context fills—has not been systematically studied prior to JarvisOS. Existing literature characterizes context degradation as an accuracy and reliability problem [16][17][18]. JarvisOS’s contribution is identifying it as a discrete security threat in privilege-escalated agentic systems, where the consequence of silently dropped constraints is not a wrong answer but unconstrained system access.

3. JARVISOS: PLATFORM AND ARCHITECTURE

JarvisOS is an Arch Linux-based, AI-native operating system designed as a controlled empirical testbed for studying the security behavior of privilege-escalated LLM agents. Its core action layer is implemented across two Rust packages—**dispatch** and **dmcp**—that together form a composable MCP orchestration layer, supported by the **contextor** memory backend and the Python-based **jarvis** application layer. Unlike conventional operating systems that treat AI as an application layer, JarvisOS integrates the LLM as a first-class system component—capable of invoking tools, managing files, executing processes, and operating with escalating levels of system privilege.

The system is built on a seven-script modular build pipeline that transforms a base Arch Linux ISO into a bootable AI-native operating system with KDE Plasma 6 on Wayland. Each script is responsible for a discrete layer of the system—from base package installation and Ollama inference setup to MCP orchestration layer configuration and desktop environment integration—making the build process reproducible, auditable, and independently extensible by contributors. Local LLM inference is handled by Ollama, though the architecture also supports API-based LLMs, allowing the platform to be used with any model backend.

dispatch is a signal-driven parallel task orchestrator built on the Tokio async runtime. Its design philosophy—*one brain, many hands*—captures the core architectural principle: a single LLM instance acts as the sole decision maker, while multiple MCP servers execute operations concurrently as workers.

When the user issues a request requiring tool use, the LLM produces a structured task list and submits it to dispatch via the MCP protocol. dispatch immediately spawns each task as an independent asynchronous operation, assigns each a unique internal PID, and pushes an INIT signal for each into a rolling signal window. The LLM then goes idle—it does not poll, does not hold context with pending tool results, and remains fully available for normal conversation. dispatch wakes the LLM only when a signal arrives: a task completes (EXIT), exceeds its reminder threshold (REMIND), or is acted upon by the user (WAIT, KILL). The LLM’s working memory at any wakeup point is a bounded rolling window of the last twenty signal entries, from which it reconstructs system state and decides the next action. This architecture achieves multi-agent-level parallelism without loading multiple LLM instances, and keeps the LLM’s context size bounded and predictable regardless of how many tasks have run.

dmcp is the MCP server lifecycle manager on which dispatch depends. It handles discovery, installation, configuration, invocation, and removal of MCP servers at both user scope and system scope, following XDG Base Directory conventions. System-scope operations require privilege elevation via PolicyKit. dmcp can also run as an MCP server itself, exposing its management and dispatch capabilities as tools callable by the LLM. The two packages are complementary and non-redundant: dmcp manages what servers exist and how to invoke them; dispatch orchestrates concurrent calls to those servers and routes signals back to the LLM.

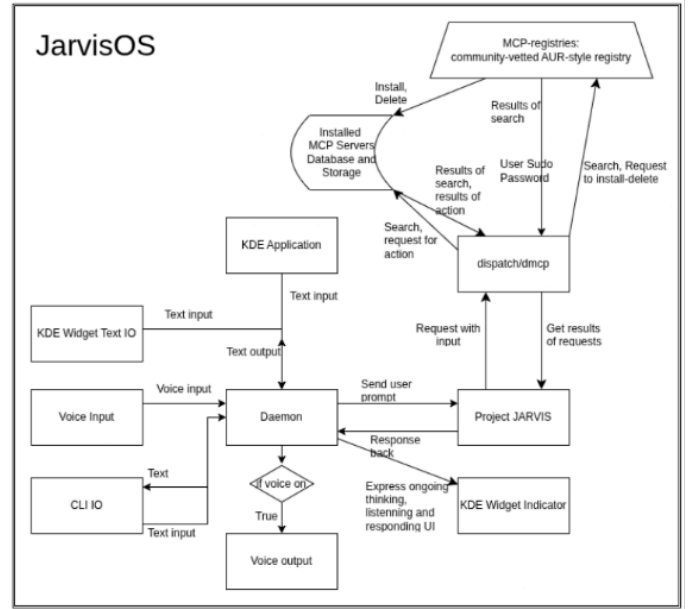


Figure 2: High-level architecture of JarvisOS showing the relationship between the build pipeline, inference layer, application routing, and the dispatch/dmcp orchestration stack.

The **jarvis** application layer implements a two-level hierarchical execution architecture. The ROOT level handles all direct user-facing interaction: standard natural language conversa-

tion, memory operations via contextor (store, recall, and vector similarity search over past sessions), session management, and routing decisions. When a user input requires tool invocation, ROOT routes it to the DISPATCH sub-chain; when it can be answered from conversation history or memory alone, ROOT responds directly without entering the sub-chain. The DISPATCH sub-chain is a sequential planning and execution pipeline: the LLM first plans which tools are needed, performs discovery to identify the relevant MCP servers via dmcp’s vector index, installs missing servers if necessary, constructs and submits the task list to the dispatch orchestrator, and waits for the signal-driven result cycle to complete before returning a summary to ROOT. The sub-chain enforces a configurable maximum depth to prevent unbounded recursive dispatch.

contextor is a persistent Rust binary that serves as the system’s memory backend. It accepts newline-delimited JSON commands, exposing operations for storing memory entries with pre-computed embedding vectors, recalling recent entries by theme, performing cosine-similarity vector search over stored conversation history, managing chat sessions with rolling summaries, and running retention-based pruning. contextor’s vector search operates over conversation memory, which is a distinct data domain from dmcp’s tool discovery index. The two vector search systems coexist and serve complementary purposes.

Tool discovery in JarvisOS is implemented in dmcp. dmcp maintains a local vector index that stores pre-computed embedding vectors at both server level and individual tool level. The search strategy is adaptive based on catalog size: when the number of visible MCP servers is below a threshold, keyword search is used—it is computationally cheaper and sufficient for small catalogs. When the catalog exceeds that threshold, dmcp switches to embedding-based cosine similarity search, which handles semantic disambiguation at scale. Both paths return results from the same underlying MCP registry. At the jarvis application layer, the embedding component generates query vectors from user intent text at dispatch planning time and passes them to dmcp to retrieve the semantically most relevant servers and tools.

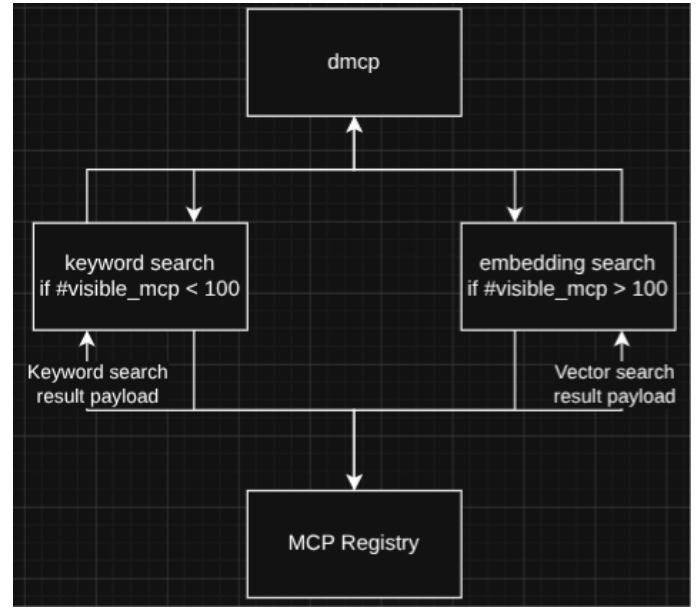


Figure 3: dmcp semantic tool discovery pipeline. Search strategy is adaptive: keyword search for small catalogs, embedding-based cosine similarity for large ones. Both paths return results from the community-vetted MCP registry.

The **Cryptographic Boundary Protocol** provides a structural mechanism for tagging the provenance of MCP server output as it enters the signal window. When the dispatch orchestrator receives a result from an MCP server, it generates a six-hexadecimal-character provenance nonce for that task. The nonce is produced by a Splitmix64 bit-mixing function that combines three inputs—the task PID multiplied by a fixed prime constant, wall-clock nanoseconds at completion time, and an atomically incremented session counter—ensuring uniqueness across concurrent tasks. On successful task completion, the full MCP server output is stored out-of-band in the orchestrator’s outputs map, accessible only through an explicit `get_output` tool call; the EXIT signal visible to the LLM carries only the nonce prefix and a status code. On task failure, error content is included in the signal window but is structurally delimited using the nonce as a tag boundary. This design achieves a structural separation between the instruction plane (the signal window) and the data plane (MCP server output stored out-of-band). Successful MCP output does not flow into the LLM’s active context unless explicitly requested, which is the most direct architectural defense against injection: content that is not in context cannot be acted upon.

The **TLA (Threat Level Access)** system implements dynamic, context-aware privilege levels ranging from guest to kernel, enforced at the operating system level. Privilege levels are assigned per session and per operation, and escalation requires explicit user confirmation. This ensures that no MCP server or LLM action can acquire privileges beyond what the current TLA level permits, regardless of what the model or a malicious server requests.

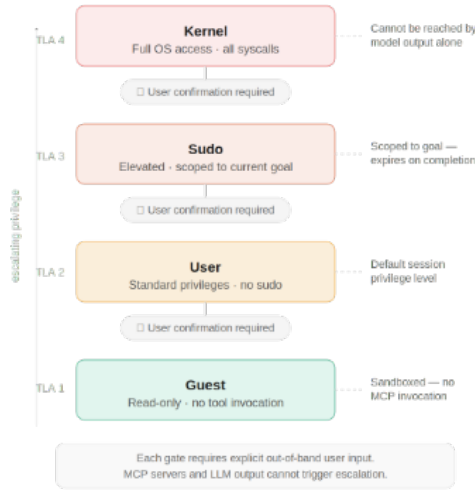


Figure 4: TLA (Threat Level Access) privilege escalation model. Each level transition requires explicit out-of-band user confirmation and cannot be triggered by model output or MCP server response alone.

Finally, JarvisOS maintains a community-vetted **AUR-style MCP registry**—modeled on the Arch Linux User Repository proofread model. Third-party MCP servers must pass community review before being listed in the registry. Reviewers inspect server code, declared capabilities, and behavior before approving listing. This supply-chain control operates upstream of the runtime search engine, ensuring that malicious or deceptive servers are filtered before they are ever discoverable by the tool search engine.

Table 1. Tools exposed by dmcp when operating as an MCP server (dmcp serve), as implemented in src/serve.rs.

Tool	Description
list_servers	List all installed MCP servers, filterable by user scope and/or system scope
get_server_info	Return detailed metadata for a server by ID, including name, version, scope, and transport types
install_server	Fetch a server from the configured registry by ID and install it to user or system scope
uninstall_server	Remove an installed MCP server by ID
set_config	Set a configuration key-value pair for an installed server, persisted to its manifest
list_server_tools	Enumerate the tools exposed by a given installed server, including names, descriptions, and input schemas
call_server_tool	Invoke a named tool on an installed server with an optional JSON argument object
dispatch_tasks	Dispatch a list of MCP tool calls for concurrent execution; returns a list of task PIDs for tracking
get_task_status	Return status and output for completed or failed tasks; optionally includes the rolling twenty-entry signal log

kill_task	Abort a running dispatched task by PID
-----------	--

3.1 Identified Threats and Mitigations

Through designing, building, and operating JarvisOS, we empirically identified six security threats that emerge when LLMs are granted elevated system privileges. Each threat was observed through direct system operation and is addressed by one or more architectural components described in Section 3. We present each threat with its attack vector, observed behavior, and mitigation.

Threat	User	Sudo	Web	Primary mitigation
Malicious MCP servers Backdoors in third-party servers	●	●	●	Community-vetted AUR-style registry
Prompt injection Malicious instructions in MCP output	●	●	●	Cryptographic Boundary Protocol (dispatch)
Misleading MCP server usage Deceptive or ambiguous tool descriptions	●	●	●	Registry vetting + structured tool schema
Unauthorized sudo requests via MCP Server requests escalation at runtime	○	●	●	TLA system + PolicyKit enforcement
Sudo capability exploitation Privilege used beyond intended scope	○	●	●	TLA + goal-scoped confirmation step
Bloated context Security constraints silently dropped	●	●	●	dispatch rolling window + context pruning

● Threat surface active at this stage ○ Not applicable at this stage

Figure 5: Threat taxonomy showing each of the six identified threats, the privilege escalation stage at which it surfaces, and the primary architectural mitigation.

3.1.1 Malicious MCP Servers

Attack vector: A third-party MCP server contains malicious code—backdoors, data exfiltration routines, or destructive commands—that executes when the LLM invokes any of its tools. The LLM has no intrinsic mechanism to verify that a server’s behavior matches its declared description, and will invoke tools it has been given access to without suspicion.

Observed behavior: When connected to an unverified server, the LLM invokes its tools in response to legitimate user requests, at which point the server’s malicious payload executes silently. The user sees a normal interaction; the server acts in the background.

Mitigation: JarvisOS addresses this threat at the supply-chain level through its community-vetted AUR-style MCP registry. Third-party server authors submit their servers for community review before they are listed. Reviewers inspect server code, declared capabilities, and behavior before approving listing. Only registry-approved servers are discoverable through dmcp’s tool search engine, meaning a malicious server cannot be loaded unless it has passed community review. This mirrors the proven governance model of the Arch Linux User Repository and operates entirely upstream of runtime—the threat is neutralized before the LLM ever sees the server.

3.1.2 Prompt Injection

Attack vector: A malicious actor embeds instructions inside content that an MCP server returns to the dispatch orchestrator

—for example, inside a file a server reads, a web page it fetches, or an API response it relays. The LLM, receiving this content in its context, may be unable to distinguish the embedded instructions from legitimate system signals and may act on them.

Observed behavior: The LLM follows instructions embedded in MCP server responses, potentially performing actions the user never requested—including privilege escalation, data exfiltration, or destructive file operations—while presenting a normal-looking interaction to the user.

Mitigation: JarvisOS addresses this threat through the Cryptographic Boundary Protocol implemented directly in the dispatch orchestrator. When an MCP server task completes, dispatch generates a six-hexadecimal-character provenance nonce using a Splitmix64 bit-mixing function that combines the task PID, wall-clock nanoseconds at completion time, and an atomically incremented session counter. On successful task completion, the full MCP server output is stored out-of-band and is not injected into the signal window at all; the EXIT signal visible to the LLM carries only the nonce prefix and a status code. The LLM must make an explicit `get_output` tool call to retrieve output. The community-vetted MCP registry provides a complementary upstream defense by ensuring unreviewed servers cannot be discovered through the tool search engine.

3.1.3 Misleading MCP Server Usage

Attack vector: An MCP server misrepresents its functionality through vague, ambiguous, or deliberately deceptive tool descriptions. The LLM, relying on these descriptions to select the appropriate tool for a given request, selects the wrong server—either invoking unintended functionality or exposing itself to a server designed to attract invocation through misleading descriptions.

Observed behavior: The LLM invokes a server whose declared description does not accurately reflect its behavior, resulting in unintended actions, incorrect outputs, or covert malicious execution dressed as legitimate tool use.

Mitigation: This threat is handled at two levels. First, at the registry level: the community review process that vets servers for malicious code also evaluates the accuracy and clarity of tool descriptions. Servers with misleading or ambiguous descriptions are rejected or required to revise before listing. Second, at the architecture level: JarvisOS’s structured tool-description format—independently developed in October–November 2025—enforces a standardized schema for declaring tool capabilities, inputs, and expected behaviors, reducing the surface area for ambiguity and making description-based deception easier to detect during review.

3.1.4 Unauthorized Sudo Requests via MCP

Attack vector: An MCP server—whether malicious or simply poorly scoped—attempts to invoke system commands that require elevated privileges beyond what the current session has been granted. In a conventional LLM agent setup, the model

may comply with such requests if it has been given sudo access at any point in the session, or attempt to request escalation on the server’s behalf.

Observed behavior: An MCP server constructs a tool call that requires sudo, either directly requesting privilege escalation from the LLM or embedding sudo commands in its tool outputs. Without enforcement mechanisms, the LLM may execute the request or attempt to acquire the necessary privilege.

Mitigation: The TLA (Threat Level Access) system enforces privilege levels at the operating system level on a per-session, per-operation basis. Every tool invocation is evaluated against the current TLA level before execution. A request that requires a privilege level above the current TLA is blocked at the OS level, regardless of what the LLM or the MCP server requests. Privilege escalation requires explicit, out-of-band user confirmation—it cannot be triggered by model output or server response alone. System-scope dmcp operations independently require PolicyKit elevation, providing a second enforcement layer at the package management level.

3.1.5 Sudo Capability Exploitation

Attack vector: Once sudo access has been legitimately granted within a session—for example, for a specific maintenance task—the LLM or a compromised MCP server exploits that elevated privilege to perform actions beyond the originally intended scope. Because the privilege was legitimately granted, no permission error is triggered, and the exploitation may go undetected.

Observed behavior: With sudo available in the session context, the LLM takes actions far outside the original task—modifying system files, installing packages, altering configurations—because nothing in a conventional setup prevents it from doing so once the privilege exists.

Mitigation: JarvisOS addresses this through a combination of the TLA system and its goal-scoped workflow confirmation mechanism. Sudo access, when granted, is scoped to the current goal—not the entire session. Any action that falls outside the defined goal scope triggers a user confirmation step before execution. This means that even with elevated privilege active, the LLM cannot silently expand its actions beyond what the user originally authorized. The TLA level is also logged and auditable, providing a post-hoc record of what privilege was active and when.

3.1.6 Bloated Context

Attack vector: As a session progresses, the LLM’s context window accumulates conversation history, MCP server responses, tool outputs, and system state—eventually approaching or exceeding the model’s effective context capacity. When this occurs, the model’s attention mechanism begins to deprioritize earlier content, including security constraints and behavioral guidelines established in the system prompt at the

start of the session. The model does not notify the user that this has happened.

Observed behavior: In extended sessions with heavy tool use, the LLM begins to ignore previously established constraints—such as restrictions on which directories it may modify, or instructions to require confirmation before destructive operations—and behaves as though those constraints were never set. This is not a discrete failure event; it is a gradual degradation that is difficult to detect in real time.

Mitigation: JarvisOS addresses this through two complementary mechanisms. First, dispatch’s bounded rolling signal window architecture ensures that MCP task results do not accumulate in the LLM’s context—only the last twenty signal entries are presented at wakeup, keeping context size predictable regardless of how many tasks have run. Second, the contextor memory backend actively manages the LLM’s working context, maintaining only the information relevant to the current goal or conversation turn via retention-based pruning. Security-critical constraints are treated as high-priority context elements and preserved through context refreshes. Together, these mechanisms prevent the context from bloating to the point where early constraints become effectively invisible to the model.

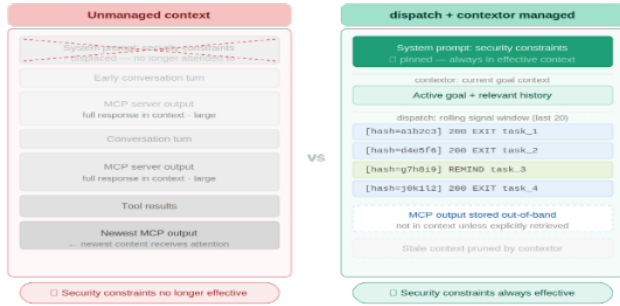


Figure 6: Bloated Context threat illustration. Left: unmanaged context where security constraints are displaced over time. Right: dispatch’s bounded rolling signal window combined with contextor pruning, preserving security constraints as high-priority elements.

4. CONCLUSION

JarvisOS was developed as both an AI-native operating system and an empirical security testbed for studying privilege-escalated LLM agents. The work presented in this paper demonstrates that securing such agents requires more than prompt-level behavioral constraints; it requires architectural mechanisms that enforce boundaries between instructions, data, tools, privileges, and persistent security state.

A recurring finding across this work is that prompt-level defenses are insufficient on their own. Instructions embedded in a system prompt—whether warnings against injection, restrictions on privileged operations, or behavioral guardrails—are

vulnerable to being circumvented, ignored, or silently degraded as context fills. Effective security for privilege-escalated LLM agents requires architectural enforcement: separation of the data and instruction planes at the protocol level, privilege controls enforced at the OS level, and context management that treats security constraints as first-class persistent state rather than passive instructions.

We also note that several threat classes documented here were not present in prior literature at the time of JarvisOS’s development. Bloated Context in particular—the silent degradation of security constraints as context window capacity is exhausted—has been studied extensively as a reliability and accuracy problem, but its implications as a discrete security threat in agentic systems had not been identified or addressed prior to this work. JarvisOS’s architectural response—storing MCP output out-of-band so it never enters the LLM’s context unless explicitly requested—represents a stronger mitigation than prompt-level warnings: it removes the attack surface entirely rather than attempting to defend against it after the fact.

Finally, JarvisOS implemented its structured MCP tool-description architecture in October–November 2025, predating its independent adoption by major AI platforms in early 2026. We document this not to make a priority claim for its own sake, but because it illustrates a broader pattern: the security community needs to be ahead of capability deployment, not behind it. JarvisOS was built on the premise that empirical research on AI agent security needs to happen now, in open platforms, with findings shared freely—before these systems become critical infrastructure.

4.1 Summary of Contributions

This paper makes four concrete contributions to the field of AI agent security.

First, we present a taxonomy of six empirically-identified security threats specific to privilege-escalated LLM agents: Malicious MCP Servers, Prompt Injection, Misleading MCP Server Usage, Unauthorized Sudo Requests via MCP, Sudo Capability Exploitation, and Bloated Context. Each threat was observed through direct system operation rather than theoretical analysis, and each is characterized by its attack vector, observed behavior, and escalation stage.

Second, for each identified threat class we developed and implemented an architectural mitigation—ranging from the community-vetted AUR-style MCP registry and the dispatch Cryptographic Boundary Protocol to the TLA privilege enforcement system and dynamic context management via contextor and dispatch’s bounded rolling signal window. These mitigations address threats at the protocol, OS, and memory management levels, rather than relying on prompt-level instructions to the model.

Third, JarvisOS itself—a fully functional, bootable, open-source AI-native operating system—is released as a research and

development platform for the community. The platform is designed to be reproducible, auditable, and extensible, enabling other researchers to build on, stress-test, and extend this work.

Fourth, we document a structured MCP tool-description architecture independently developed in JarvisOS in October–November 2025, predating its adoption by major AI platforms in early 2026. This contribution establishes that the security research community identified and implemented this design pattern before it became standard practice in commercial deployments.

4.2 Future Work

JarvisOS is an active and ongoing project. The platform is being prepared for broader community use, with documentation, contribution guidelines, and a public MCP registry infrastructure that will allow third-party developers to submit and vet MCP servers under the proofread model described in this paper.

Fine-tuning work is planned using the labeled provenance-nonce dataset generated as a byproduct of dispatch’s output-tagging mechanism. A LoRA/QLoRA fine-tune of Llama 3.1 8B using the NVIDIA NeMo Framework is in preparation, with the resulting model and dataset intended for public release on HuggingFace.

Continued empirical evaluation across the full six-threat taxonomy is planned, moving from the framework and platform contribution presented here toward quantitative attack reproduction results—measuring attack success rates, detection rates, and mitigation effectiveness under controlled conditions.

JarvisOS is released as open-source software and the community is invited to contribute code, MCP servers, threat findings, and tooling. The project is available at <https://github.com/YakupAtahanov/Project-JARVIS>.

REFERENCES

- [1] “Prompt Injection Attacks on Large Language Models: A Survey of Attack Methods, Root Causes, and Defense Strategies,” *ScienceDirect*, Feb. 2026. [Online]. Available: <https://www.sciencedirect.com/org/science/article/pii/S1546221826001384>
- [2] E. Greshake et al., “Benchmarking and Defending Against Indirect Prompt Injection Attacks on Large Language Models,” *arXiv:2312.14197*, 2023.
- [3] B. Huang and F. N. de Paula, “Defend LLMs Through Self-Consciousness,” *arXiv:2508.02961*, 2025.
- [4] A. Dehghantanha and S. Homayoun, “SoK: The Attack Surface of Agentic AI—Tools, and Autonomy,” *arXiv:2603.22928*, 2025.
- [5] J. Kim, W. Choi, and B. Lee, “Prompt Flow Integrity to Prevent Privilege Escalation in LLM Agents,” *arXiv:2503.15547*, 2025.
- [6] “Evaluating Privilege Usage of Agents on Real-World Tools,” *arXiv:2603.28166*, 2025.
- [7] “The Dark Side of LLMs: Agent-based Attacks for Complete Computer Takeover,” *arXiv:2507.06850*, 2025.

- [8] Z. Zou et al., “PoisonedRAG: Knowledge Corruption Attacks to Retrieval-Augmented Generation of Large Language Models,” *USENIX Security*, 2025. Available: <https://arxiv.org/pdf/2402.07867>
- [9] “Secure Retrieval-Augmented Generation against Poisoning Attacks (RAGuard),” *arXiv:2510.25025*, 2025.
- [10] “Semantic Tool Discovery for Large Language Models: A Vector-Based Approach to MCP Tool Selection,” *arXiv:2603.20313*, 2026.
- [11] H. Errico, J. Ngiam, and S. Sojan, “Securing the Model Context Protocol (MCP): Risks, Controls, and Governance,” *arXiv:2511.20920*, 2025.
- [12] “Toward Understanding Security Issues in the Model Context Protocol Ecosystem,” *arXiv:2510.16558*, 2025.
- [13] “MCPGuard: Automatically Detecting Vulnerabilities in MCP Servers,” *arXiv:2510.23673*, 2025.
- [14] OX Security, “The Mother of All AI Supply Chains: Critical, Systemic Vulnerability at the Core of Anthropic’s MCP,” Apr. 2026. Available: <https://www.ox.security/blog/the-mother-of-all-ai-supply-chains-critical-systemic-vulnerability-at-the-core-of-the-mcp/>
- [15] SentinelOne, “Model Context Protocol (MCP) Security: Complete Guide,” Mar. 2026. Available: <https://www.sentinelone.com/cybersecurity-101/cybersecurity/mcp-security/>
- [16] AWS Security Blog, “Context Window Overflow: Breaking the Barrier,” Jul. 2024. Available: <https://aws.amazon.com/blogs/security/context-window-overflow-breaking-the-barrier/>
- [17] N. Paulsen, “LLM Context Window Limitations,” 2025, cited in Atlan, Apr. 2026. Available: <https://atlan.com/know/llm-context-window-limitations/>
- [18] “Not All Needles Are Found: How Fact Distribution and Prompts Shape Literal Extraction, Logical Inference, and Hallucination Risks in Long-Context LLMs,” *arXiv:2601.02023*, 2026.
- [19] “Taming OpenClaw: Security Analysis and Mitigation of Autonomous LLM Agent Threats,” *arXiv:2603.11619*, 2026.
- [20] Y. Atahanov, “Project-JARVIS (JarvisOS),” GitHub, 2025. [Online]. Available: <https://github.com/YakupAtahanov/Project-JARVIS>

Additional notes:

1. For a security research paper, reviewers will expect: Threat Model — explicitly defining the adversary’s capabilities, goals, and assumptions; Limitations — the paper’s constraints and what the system does not defend against; Evaluation/Results — even qualitative or observational evidence presented in a structured way. The paper jumps directly from §3 (Architecture) to §4 (Conclusion), with no evaluation section. Even if quantitative results are forthcoming, a structured discussion of qualitative observations from operating the system would strengthen the empirical claims.
2. The context window FIFO description in §2.5 (“first-in, first-out buffer”) is a simplification — context truncation behavior varies by implementation and is not universally strict FIFO. Softening this to “earlier tokens are disproportionately deprioritized” would be more technically accurate.
3. Figure quality appears low-contrast and may not meet IEEE figure resolution requirements — the images should be exported at higher resolution with legible labels.